DATABASE MANAGEMENT SYSTEMS | BTCS501-C

DBMS Complete Notes

Module 1: Database System Architecture & Module 2: Relational Query Languages

University: S B S State University, Ferozepur

Department: Computer Science & Engineering

Course Code: BTCS501-C | **Credits:** 3L : 0T : 0P (3 Credits)

Prepared for: B.Tech CSE, Roll No. 24585

Covers: CO1 (design/write relational expressions) & CO2 (ER modelling, SQL, normalization)

Table of Contents

MODULE 1 — Database System Architecture

1. Introduction to DBMS — Data, Information & Database
2. File Processing System & its Limitations
3. Database Approach, DBMS Advantages & Disadvantages
4. Three-Level (Three-Tier) ANSI-SPARC Architecture
5. Schemas, Instances & Data Independence
6. Data Models (Hierarchical, Network, Relational)
7. E-R Model — Entities, Attributes, Relationships, Cardinalities
8. Relational Data Model — Terminology & Keys
9. Integrity Constraints
10. Normalization (1NF, 2NF, 3NF, BCNF)
11. Module 1 — Practice Questions (2/4/8 Marks + MCQs)

MODULE 2 — Relational Query Languages

1. Introduction — Procedural vs Non-Procedural Languages
2. Relational Algebra — Fundamental & Derived Operations
3. Relational Calculus — Tuple & Domain Relational Calculus
4. Relational Algebra vs Relational Calculus
5. SQL — DDL, DML, DCL, TCL commands with examples
6. SQL Clauses, Joins & Aggregate Functions
7. Module 2 — Practice Questions (2/4/8 Marks + MCQs)

MODULE 1

Database System Architecture

Syllabus: Data Abstraction, Data Independence, DDL, DML, Data Models — E-R model, network model, relational and object-oriented data models, integrity constraints, data manipulation operations.

1. Introduction to DBMS

1.1 Data vs Information

Data is a collection of raw, unprocessed facts and figures that, by themselves, carry no meaning — names, numbers, symbols. **Information** is data that has been processed, organized or structured so that it becomes meaningful and useful for decision-making.

> **Real-life example:** The number **240343** alone is meaningless — it could be a date of birth, part number, or population count (Data). But once written as **24-03-1943 (Date of Birth)**, it becomes meaningful (Information). Similarly, a student's marks in five subjects are data; the final **report card** showing percentage and result is information.

Data	Information
Raw, unorganized facts and figures	Processed and organized form of data
Example: 23	Example: Age = 23
Not significant to business by itself	Significant and useful for decisions
Atomic-level pieces	Collection of related data items
Does not help decision making	Helps in decision making

1.2 Database

A **database** is an organized, related collection of data stored so that it can be easily accessed, managed and updated. Just as a dictionary organizes words alphabetically so any word can be found quickly, a database organizes records so that information can be retrieved efficiently.

> **Real-life example:** Your phone's **Contacts app** is a database — each contact (name, number, email) is a record, and the app lets you add, search, edit and delete contacts instantly instead of flipping through a paper diary.

1.3 Database Management System (DBMS)

A **DBMS** is software that enables users to define, create, maintain and control access to a database. It sits between the user/application programs and the physical data, hiding the complexity of storage.

> **Examples of DBMS software:** MySQL, Oracle, MS SQL Server, IBM DB2, PostgreSQL, MS Access.
> **Real-world DBMS applications:** Banking (ATM transactions), Airline reservation systems, University record systems, Online shopping (Amazon/Flipkart inventory & orders), Library management systems.

1.4 Traditional File Processing System

Before DBMS, organizations stored data in separate flat files, each maintained independently by different departments and accessed through custom application programs written in languages like COBOL or C.

> **Real-life example:** In a college, the General Office keeps a student file, the Library keeps a separate issue-register file, the Accounts Office keeps a separate fee file, and the Hostel keeps its own file — even though Roll No., Name and Class are repeated in all four.

Limitations of the File Processing System

Problem	Explanation with Example

Data Redundancy & Inconsistency	Same data (e.g. student address) is duplicated in multiple files. If a student shifts city and only the Library file is updated but not the Accounts file, the two files disagree — this is *inconsistency*.
Data Isolation	Data is scattered across files in different formats (C file vs COBOL file), making it hard to write one program that pulls related data from several files together.
Data Dependence	The physical storage format is hard-coded into the application program. Even a small change (e.g. increasing phone number length) requires rewriting every program that touches that file.
Integrity Problems	Constraints such as "attendance must be between 0-100%" must be enforced separately, in code, inside every single application — easy to miss in one of them.
Atomicity Problems	If a bank transfer program crashes halfway (debit done, credit not done), the file system has no automatic way to roll back the half-finished operation.
Concurrent Access Anomalies	Two clerks updating the same record (e.g. railway ticket seat) at the same time can overwrite each other's changes — e.g. two passengers booking the last seat simultaneously.
Security Problems	It is difficult to enforce that only the Accounts Office can see salary data, since every application has direct, unrestricted file access.

1.5 The Database Approach

To overcome these problems, the **database approach** stores all organizational data in one shared, centrally-controlled repository, described using metadata (data about data), and accessed uniformly by all applications through the DBMS.

> **Real-life example:** Instead of four separate college files, one central **Student database** stores Roll No., Name, Class once (in a "General Office" table); the Library, Hostel and Accounts tables simply refer to that Roll No. as a foreign key and fetch Name/Class by joining — eliminating repeated storage of the same facts.

Characteristics of a Database

Property	Meaning
Shared	Same data used by many users/applications at once
Persistence	Data continues to exist beyond the program run that created it
Integrity/Validity	Data must correctly reflect the real world
Security	Protected from unauthorized access
Consistency	Related data values agree with each other
Non-redundancy	Same fact is not stored needlessly more than once
Independence	Changes at one level do not disturb other levels

1.6 Advantages of DBMS

Advantage	Real-life Illustration
Controls Redundancy	College Roll No./Name stored once centrally; Library & Hostel just reference it by Roll No.
Enforces Integrity	A rule like "Class must be one of BA/BCA/BBA..." is defined once in the database and applies to every application automatically — entering "MCA" is rejected everywhere.
Avoids Inconsistency	Updating a student's address once in the central table means every department instantly sees the new address (propagation of update).
Data Sharing	Airline booking systems let thousands of travel agents see the same live seat availability.
Restricting Unauthorized Access	A bank teller can view/update balances but only the branch manager can view salary details, via login-based access control.
Backup & Recovery	If a server crashes mid-transaction, the DBMS restores the database to the last consistent state (as in ATM transaction failures — money is never "lost in between").

Standards Enforcement	A company can enforce a uniform date format (DD-MM-YYYY) across every department's data entry screens.

1.7 Disadvantages of DBMS

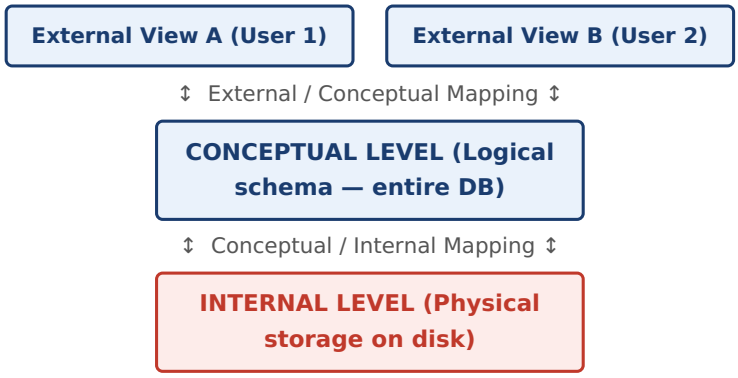
- **Complexity** — DBMS software is large and requires trained administrators.
- **Cost** — Licensing (Oracle, SQL Server) plus hardware plus training is expensive.
- **Performance overhead** — a DBMS is general-purpose, so a hand-written file program for one narrow task can sometimes be faster.
- **Higher impact of failure** — since everything is centralized, a DBMS crash halts every department at once (single point of failure).
- **Conversion cost** — migrating old file-based systems into a DBMS and retraining staff is costly and time consuming.

1.8 File System vs DBMS — Comparison

File Management (e.g. C/COBOL program)	Database Management (e.g. Oracle/MySQL)
Small, simple systems	Large, complex systems
Relatively cheap	Relatively expensive
Files are independent flat files	Data organized as related tables
High redundant data	Redundancy reduced/controlled
Chances of inconsistency	Consistent, centrally managed
Isolated data, hard to combine	Data can be easily shared/joined
Integrity left to the programmer	Rigorous, built-in integrity checking
No/weak security	Rigorous, role-based security
Primitive, manual backup	Complex, automated backup/recovery
Usually single user	Supports multiple concurrent users

2. Three-Level (Three-Tier / ANSI-SPARC) Database Architecture

Proposed by ANSI-SPARC (1975), the goal of the three-level architecture is to separate **how each user sees the data** from **how the data is physically stored**, so that changes at one level do not disturb the others. It has three layers:



2.1 External Level (View Level)

This is the level closest to individual users. It shows each user only the part of the database relevant to them, hiding the rest.

> **Real-life example (College Database):** A **Library clerk's** view shows Roll No., Name, Book No., Issue Date, Return Date. An **Accounts clerk's** view of the very same database instead shows Roll No., Name, Fee, Balance. Neither sees the other's fields, even though both views are drawn from one common database.

2.2 Conceptual Level (Logical Level)

This is the community/DBA's view — it describes **what** data is stored (all entities, attributes, relationships and constraints) for the whole organization, independent of physical storage details or any one user's needs.

> **Example:** The conceptual schema defines fields like RollNo: Number(6), Name: Character(20), Fee: Number(7,2) — without saying which disk sector they occupy.

2.3 Internal Level (Physical/Storage Level)

This is the lowest level, describing **how** data is actually stored on the hardware — file organization, indexing, byte offsets, compression, and access paths.

> **Example:** RollNo field → Type=Byte(6), Offset=0; Name field → Type=Byte(20), Offset=6 — the exact physical layout on disk.

2.4 Schemas & Instances

Term	Meaning
Schema	The overall design/blueprint of the database (structure) — changes rarely.
Instance	The actual data present in the database at a given moment in time — changes constantly (every INSERT/UPDATE/DELETE).
External Schema	Description of one user's view.
Conceptual Schema	Description of the entire logical database (one per DB).
Internal Schema	Description of physical storage structures (one per DB).

> **Analogy:** Schema is like a blank *mark-sheet template* (fields: Roll No, Name, Marks); an instance is a *filled-in mark-sheet* for a particular student on a particular day — the template stays the same while the filled values change every exam.

2.5 Data Independence

Data independence is the capacity to change the schema at one level without needing to change the schema at the next higher level. This is precisely what the three-level architecture is designed to provide.

Type	Meaning	Example
Logical Data Independence	Ability to change the conceptual schema without affecting external schemas/application programs.	Adding a new column "Email" to the Student table does not break the existing Library clerk's screen, which never referenced Email.
Physical Data Independence	Ability to change the internal (storage) schema without affecting the conceptual schema.	Migrating the database from a hard disk to SSD storage, or changing from sequential file storage to B-tree indexing, does not require rewriting any application queries.

> **Note:** Logical data independence is harder to achieve than physical data independence, because application programs are more tightly tied to the logical structure of data than to its physical storage.

3. Data Models

A **data model** is an abstraction that represents real-world objects and their associations, concentrating on the essential structure while hiding unnecessary detail. It defines how data is logically organized, connected and manipulated. The three classical record-based data models are the **Hierarchical**, **Network** and **Relational** models.

3.1 Hierarchical Data Model

One of the earliest models (late 1950s, e.g. IBM's IMS). Data is organized as a **tree structure**: every record (child) has exactly **one parent**, forming 1:1 and 1:N relationships only.

Real-life example: A company's organization chart — one CEO has many managers under them, each manager has many employees under them. Each employee reports to exactly one manager (a strict parent-child tree). Similarly, a College → Department → Student tree.

Operation	Behaviour / Anomaly
Insert	Cannot insert a child record without first having its parent (e.g. cannot add a Student without an existing Department).
Update	If data is repeated under multiple parents, every occurrence must be updated separately — inconsistency risk.
Delete	Deleting a parent automatically deletes all its child records (e.g. deleting a Department deletes all its Students).
Retrieve	Asymmetric — searching top-down is fast, but going child→parent or across branches is awkward.

Limitation: Cannot naturally represent Many-to-Many relationships — e.g. an employee working in two departments cannot be modelled cleanly.

3.2 Network Data Model

The Network model replaces the rigid tree with a more general **graph**, allowing a record to have **more than one parent** — thus supporting 1:1, 1:N **and** M:N relationships. In network terminology, a relationship is called a **set**, made of an **owner** record and one or more **member** records, connected using pointers/chains.

Real-life example: Supplier–Part–Shipment scenario — a supplier (e.g. S1) supplies many parts, and a part (e.g. P1) is supplied by many suppliers. A "connector" record links each Supplier–Part pair and stores the shipped quantity. This M:N relation is naturally expressed as a network graph, unlike a hierarchy.

Operation	Behaviour
Insert	A new supplier with no parts yet can be inserted independently (pointer just loops to itself) — not possible in the hierarchical model.
Update	Simple — each fact (e.g. supplier's city) appears only once, so update needs no search and causes no inconsistency.
Delete	Deleting one part's record does not affect its suppliers — only the connecting pointers/connector are removed.
Retrieve	Symmetric (finding suppliers-of-a-part and parts-of-a-supplier use similar algorithms) but complex, since many pointer chains must be traversed.

Conclusion: Network model removes insert/update/delete anomalies of the hierarchical model, but its heavy reliance on pointers makes it structurally complex.

3.3 Relational Data Model

Proposed by **Dr. E. F. Codd (IBM, 1970)**, this is the most widely used model today. Data is stored simply as **tables (relations)** of rows and columns; relationships between tables are expressed only through shared column values (no physical pointers).

Real-life example: A **Supplier** table, a **Part** table and a **Shipment** table (Sno, Pno, Qty) — the Shipment table links suppliers to parts purely by matching SNo and PNo values, not physical pointers, so it is easy to understand and query using plain SQL.

Data Model	Structure	Relationship Types Supported	Complexity
Hierarchical	Tree	1:1, 1:N only	Low, but rigid — insert anomalies
Network	Graph	1:1, 1:N, M:N	No anomalies, but pointer-heavy & complex
Relational	Tables (relations)	1:1, 1:N, M:N (via foreign keys)	Simple, symmetric, set-based (SQL)

(The Relational Model is covered in complete detail in Section 8 of this module, and queried using Relational Algebra/Calculus/SQL in Module 2.)

4. Entity-Relationship (E-R) Model

The E-R Model is a **top-down, graphical** approach to database design proposed by Peter Chen. It represents the real world as a set of **Entities**, their **Attributes**, and the **Relationships** among them, before converting the design into relational tables.

4.1 Entity & Entity Set

An **Entity** is a real-world object that is distinguishable from every other object — e.g. a particular student, a particular car. An **Entity Set** is a collection of similar entities that share the same attributes — e.g. the set of ALL students.

> **Example:** "Rahul, Roll No. 101" is one entity; the entity set **STUDENT** contains every student in the college.

4.2 Attributes & their Types

Attribute Type	Meaning	Example
Simple (Atomic)	Cannot be divided further	Roll_No, Age
Composite	Can be divided into sub-parts	Name → (First_Name, Middle_Initial, Last_Name); Address → (Street, City, State, Zip)
Single-valued	Holds only one value per entity	Date_of_Birth
Multi-valued	Can hold many values per entity	Phone_Number (a customer may have 2-3 numbers)
Derived	Computed from another attribute, not stored directly	Age (derived from Date_of_Birth)
Key Attribute	Uniquely identifies each entity (underlined in diagrams)	Customer_id

> **Real-life example — CUSTOMER entity:**
> **CUSTOMER** (customer_id [key], name(first_name, middle_initial, last_name) [composite], address(street(street_number, street_name, apartment_number), city, state, zip_code) [composite], phone_number [multi-valued], date_of_birth [simple], age [derived from date_of_birth])

4.3 Symbols used in E-R Diagrams

Symbol	Meaning
Rectangle	Entity Type
Double Rectangle	Weak Entity Type
Diamond	Relationship Type
Double Diamond	Identifying Relationship Type
Ellipse (oval)	Attribute
Underlined Ellipse	Key Attribute
Double Ellipse	Multivalued Attribute
Ellipse connected to sub-ellipses	Composite Attribute
Dashed Ellipse	Derived Attribute
Double line (edge)	Total Participation of an entity in a relationship
1, N labels on edges	Cardinality ratio (e.g. 1:N)

4.4 Relationship & Relationship Set

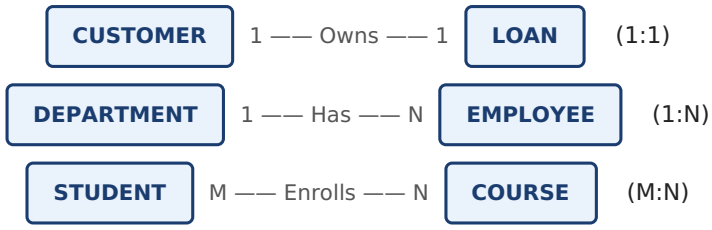
A **Relationship** is an association among two or more entities. A **Relationship Set** is a collection of similar relationships.

> **Example:** "Attishoo works in the Pharmacy Department" is one relationship instance; **WORKS_IN** is the relationship set connecting the EMPLOYEE and DEPARTMENT entity sets. A relationship set can also carry its own **descriptive attributes** — e.g. WORKS_IN has a "since" (joining date) attribute.

4.5 Mapping Cardinalities (Cardinality Ratios)

Cardinality describes how many entities of one set can relate to how many entities of another set through a relationship.

Type	Meaning	Real-life Example
One-to-One (1:1)	One entity of A relates to at most one entity of B and vice versa	One CUSTOMER *owns* one LOAN account (in a bank where each customer holds exactly one loan)
One-to-Many (1:N)	One entity of A relates to many entities of B, but each B relates to only one A	One DEPARTMENT *has* many EMPLOYEES, but each employee belongs to one department
Many-to-One (N:1)	Reverse of 1:N	Many STUDENTS *enroll in* one COURSE section
Many-to-Many (M:N)	Many entities of A relate to many entities of B	Many STUDENTS *enroll in* many COURSES (each student takes several courses; each course has several students)



4.6 Participation Constraints

Type	Meaning	Example
Total Participation	Every entity in the set MUST take part in the relationship (shown by a double line)	Every DEPARTMENT must have a manager — total participation of Department in "Manages"
Partial Participation	Only some entities take part (single line)	Not every EMPLOYEE manages a department — partial participation of Employee in "Manages"

4.7 Weak Entity Sets

A **weak entity** has no primary key of its own strong enough to uniquely identify it — it can only be identified by combining its own partial key with the primary key of its **owner (strong) entity**. A weak entity must always have **total participation** in its identifying relationship with the owner.

> **Real-life example:** A **DEPENDENT** (a policy-holder's family member) has a name (pname) that is unique only within that employee's family, not across the whole company. So DEPENDENT is identified by (pname + owning Employee's ssn) together — DEPENDENT is a weak entity, EMPLOYEE is its owner/strong entity. Deleting the Employee automatically deletes all his/her Dependents (ON DELETE CASCADE).

4.8 Generalization, Specialization & ISA Hierarchies

Concept	Meaning	Example
Specialization	Top-down: splitting a high-level entity into lower-level, more specific sub-entities	EMPLOYEE → HOURLY_EMPLOYEE / CONTRACT_EMPLOYEE
Generalization	Bottom-up: combining common features of several entities into one higher-level entity	SAVINGS_ACCOUNT + CURRENT_ACCOUNT → generalized as ACCOUNT
ISA ("is a")	Lower entity inherits all attributes of the higher	HOURLY_EMPLOYEE "is a" EMPLOYEE — inherits ssn, name,

Hierarchy	entity (like inheritance in OOP)	lot, and adds its own hourly_wages, hours_worked

- **Overlap constraint:** can an entity belong to two subclasses at once? (e.g. can Joe be both Hourly and Contract? Allowed/disallowed as per rules.)
- **Covering constraint:** must every entity of the parent belong to at least one subclass? (e.g. must every Employee be either Hourly or Contract?)

4.9 Aggregation

Aggregation treats an entire relationship set as a higher-level "abstract entity" so that it can itself take part in another relationship — used when a relationship needs to relate to a relationship.

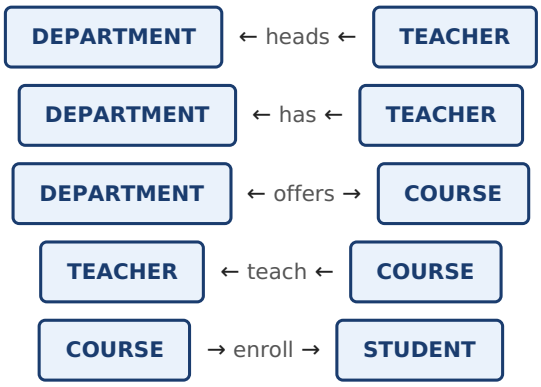
> **Example:** The relationship "TEACHER teaches COURSE" is aggregated into a single unit "Teacher-Teaches-Course", which can then participate in a further relationship "Uses" with the entity BOOK (e.g. recording which textbook is used for a particular teacher-course combination).

4.10 Steps in Designing an E-R Diagram

1. Identify the Entities from the requirement specification.
2. Find the Relationships among these entities.
3. Identify the Key (primary) attribute for every entity.
4. Identify other relevant descriptive attributes.
5. Draw the complete E-R diagram with all attributes, including the primary key.
6. Review the diagram with the business users/stakeholders.

4.11 Worked Real-Life Example — University Management System

Entities: **DEPARTMENT**(dno, dname, loc), **TEACHER**(t_code, name, mob_no), **COURSE**(c_code, c_name, credits), **STUDENT**(roll_no, name, address).



A Department *has* many teachers and one teacher *heads* the department (1:1 for heads); a Department *offers* several courses; a teacher may *teach* a course; students *enroll* in courses (M:N).

4.12 Another Example — Car Insurance Company

"*A customer owns one or more cars. Each car has zero to many recorded accidents.*"



Customer(Customer_ID, Name, Address, Phone_No, Driving_License); Car(Car_ID, Type, Data); Accident(Accident_ID, Date, Place). "Owns" is M:N (a customer can own many cars, and — for a shared/company car — a car could be linked to more than one customer); "Associated_with" links a Car to the Accidents it has been in.

5. Relational Data Model — Terminology & Keys

A **relation** is a table with rows and columns. Formal terminology:

Term	Common Name	Meaning
Relation	Table	The whole structure of rows & columns
Tuple	Row / Record	A single entry in the relation
Attribute	Column / Field	A named property of the relation
Domain	—	The set of all legal/permitted values an attribute can take
Degree	—	Number of attributes (columns) in a relation
Cardinality	—	Number of tuples (rows) in a relation

Example — EMPLOYEE relation:

Emp_Code	Name	Year
21130	Amar Jain	1
30143	Kuldeep	3
41894	Manoj	2
51207	Rita Bajaj	6

Here, **Degree = 3** (Emp_Code, Name, Year) and **Cardinality = 4** (4 rows).

5.1 Keys of a Relation

Key	Definition	Example
Super Key	Any set of attributes that can uniquely identify a tuple (may contain extra, unnecessary attributes)	{Roll_No}, {Roll_No, Name} are both super keys of STUDENT
Candidate Key	A minimal super key — no attribute can be removed without losing uniqueness	{Roll_No}, {Aadhar_No} — both could uniquely identify a student
Primary Key	The candidate key chosen by the designer to uniquely identify each row; cannot be NULL	Roll_No chosen as the Student table's Primary Key
Alternate Key	Candidate key(s) NOT chosen as the primary key	Aadhar_No (since Roll_No was chosen as primary)
Foreign Key	An attribute in one table that refers to the Primary Key of another (or the same) table, enforcing a link between them	Dept_No in STUDENT table refers to Dept_No (Primary Key) of DEPARTMENT table
Composite Key	A primary key made up of two or more attributes together	(Student_Roll_No, Course_Code) together form the key of an ENROLLMENT table

Real-life example — Supplier/Part/Shipment DB: SNo is Primary Key of SUPPLIER; PNo is Primary Key of PART; in SHIPMENT(SNo, PNo, Qty) — SNo and PNo are Foreign Keys referring back to SUPPLIER and PART respectively, and together (SNo, PNo) form the Composite Primary Key of SHIPMENT.

6. Integrity Constraints

Integrity Constraints are rules/conditions that a relation must always satisfy to remain a valid relation, ensuring that changes made by any user never break data consistency. There are three fundamental Relational Integrity Constraints:

Domain | **Key / Entity** | **Referential**

6.1 Domain Integrity Constraint

Every attribute must take values only from its defined domain (valid data type, size, null-ability, range, default, uniqueness).

Example: In an EMPLOYEE table, Age (Integer) must always be **≥ 18** — a value like `Age = -5` or `Age = 'abc'` violates the domain constraint.

6.2 Key / Entity Integrity Constraint

No two tuples in a relation can have identical values for the (primary) key attribute, and the primary key can never be NULL — this guarantees every row is uniquely identifiable.

Example: In a VEHICLE table with Reg_No as primary key, a row with `Reg_No = NULL` is rejected, and two rows can never both have `Reg_No = 'CCE-325'`.

6.3 Referential Integrity Constraint

A foreign key value must either match an existing primary key value in the referenced (parent) table, or be NULL — it can never point to a non-existent row.

Example: `CREATE TABLE Course (Course_id VARCHAR(10), Course_name VARCHAR(20), Rollno INT REFERENCES Student(Rollno));` — you cannot insert a Course row with a Rollno that does not already exist in the Student table.

6.4 Other Common SQL Constraints (Column & Table Level)

Constraint	Purpose	Example / Table Illustration
NULL	Attribute is allowed to be left empty (optional field)	Mobile_No may be NULL — e.g. row for X2 and X3 has Mobile_No = NULL
NOT NULL	Attribute cannot be left empty (compulsory field)	Emp_Id NOT NULL — every row must supply a value
UNIQUE	All values in the column must be different (NULLs allowed); a table can have several UNIQUE columns	Email column — no two employees can share the same email
PRIMARY KEY	Combination of UNIQUE + NOT NULL; uniquely identifies each row; only one per table (may be composite)	Emp_Id as Simple Primary Key; (SNo, PNo) as Composite Primary Key
FOREIGN KEY	Links to the Primary Key of another table; enforces referential integrity	Dept_No in EMPLOYEE references Dept_No in DEPARTMENT
CHECK	Validates data against a logical condition before allowing insert/update	`CHECK (Age >= 18)`
DEFAULT	Auto-fills a pre-set value into a column if the user does not supply one	`City DEFAULT 'Ferozepur'`

Illustration — Domain check applied on AGE column:

Emp_ID	Add	Age	Name
X1	VBN	21	A
X2	MTR	23	E
X3	MUB	22	D

Constraint applied: `CHECK (Age >= 18)` — Age can never be negative or less than 18.

7. Normalization

Normalization is the step-by-step process of organizing the columns and tables of a relational database to minimize **data redundancy** and eliminate **update/insert/delete anomalies**, by decomposing large tables into smaller, well-structured related tables.

7.1 Why Normalize? — Anomalies in an Unnormalized Table

Example — unnormalized STUDENT_COURSE table:

Roll_No	S_Name	Course_Id	Course_Name	Instructor
101	Aman	C1	DBMS	Dr. Rao
101	Aman	C2	OS	Dr. Sen
102	Priya	C1	DBMS	Dr. Rao

- **Insertion anomaly:** A new Course (say C3) cannot be added until at least one student registers for it, since Roll_No is part of the key.
- **Update anomaly:** If Dr. Rao is replaced, every row containing C1 must be updated — miss one row and the data becomes inconsistent.
- **Deletion anomaly:** If Priya (the only student in C1... in a smaller example) drops the course, all information about "C1 = DBMS, taught by Dr. Rao" is lost along with her row.

7.2 Functional Dependency (FD)

An attribute B is **functionally dependent** on attribute A (written A → B) if a given value of A always determines exactly one value of B.

Example: Roll_No → S_Name (each Roll No. determines exactly one student name); Course_Id → Course_Name, Instructor.

7.3 First Normal Form (1NF)

Rule: Every attribute must hold only a single, atomic (indivisible) value — no repeating groups or multi-valued/composite columns within one cell.

Before 1NF (violates atomicity):

Roll_No	Name	Phone_Numbers
101	Aman	9876543210, 9123456780

After 1NF:

Roll_No	Name	Phone_Number
101	Aman	9876543210
101	Aman	9123456780

7.4 Second Normal Form (2NF)

Rule: Must already be in 1NF, AND every non-key attribute must depend on the **whole** composite primary key — no **partial dependency** on only part of the key.

Before 2NF — key = (Roll_No, Course_Id); but S_Name depends only on Roll_No (partial dependency), and Course_Name/Instructor depend only on Course_Id:

Roll_No	Course_Id	S_Name	Course_Name	Instructor
101	C1	Aman	DBMS	Dr. Rao

After 2NF — split into 3 tables:

STUDENT
Roll_No (PK), S_Name

COURSE

Course_Id (PK), Course_Name, Instructor

ENROLLMENT
Roll_No (FK), Course_Id (FK) — composite key

7.5 Third Normal Form (3NF)

Rule: Must already be in 2NF, AND no non-key attribute may depend on another non-key attribute (no **transitive dependency**).

Before 3NF — key = Course_Id; but Instructor_Office depends on Instructor, which itself depends on Course_Id (Course_Id → Instructor → Instructor_Office is transitive):

Course_Id	Course_Name	Instructor	Instructor_Office
C1	DBMS	Dr. Rao	Room 204

After 3NF — split into 2 tables:

COURSE
Course_Id (PK), Course_Name, Instructor (FK)

INSTRUCTOR
Instructor (PK), Instructor_Office

7.6 Boyce-Codd Normal Form (BCNF)

Rule: A stricter version of 3NF — for every functional dependency A → B, A must be a **super key**. This handles rare cases in 3NF tables that still have anomalies because of overlapping candidate keys.

Example: In a table STUDENT_SUBJECT_TEACHER(Student, Subject, Teacher), suppose each Teacher teaches only one Subject (Teacher → Subject), but a Subject can be taught by several Teachers. Here Teacher determines Subject even though Teacher alone is not a candidate key — this violates BCNF. It is decomposed into TEACHER_SUBJECT(Teacher, Subject) and STUDENT_TEACHER(Student, Teacher) to remove the anomaly.

Summary Table:

Normal Form	Removes	Condition
1NF	Repeating groups / multi-valued cells	All attribute values atomic
2NF	Partial dependency	1NF + non-key attributes depend on whole key
3NF	Transitive dependency	2NF + non-key attributes depend only on the key
BCNF	Anomalies from overlapping candidate keys	Every determinant is a super key

11. Module 1 — Practice Questions

2-Mark Questions

1. Differentiate between Data and Information with an example.
2. What is Data Independence? Name its two types.
3. Define Degree and Cardinality of a relation.
4. What is a weak entity? Give one example.
5. Define Primary Key and Foreign Key.
6. What is a derived attribute? Give an example.
7. Define Referential Integrity Constraint.
8. What is the difference between a Candidate Key and a Primary Key?

4-Mark Questions

1. Explain the limitations of the File Processing System with a suitable example.
2. Describe the Three-Level (ANSI-SPARC) architecture of DBMS with a diagram.
3. Differentiate between the Hierarchical, Network and Relational data models.
4. Explain Total and Partial participation constraints with examples.
5. Explain Generalization, Specialization and ISA hierarchy with an example.
6. Explain 1NF, 2NF and 3NF with a short example of each.
7. What are Domain, Entity and Referential integrity constraints? Explain briefly with examples.
8. Differentiate between Logical and Physical Data Independence.

8-Mark Questions

1. What is a DBMS? Explain its advantages and disadvantages over the traditional file processing system in detail.
2. Explain the Three-Level Architecture of DBMS in detail with a neat diagram, and explain the two types of mapping between levels.
3. Construct an E-R diagram for a "Hospital Management System" with Patients, Doctors and Tests. Explain all entities, attributes, relationships and cardinalities used.
4. What is Normalization? Explain 1NF, 2NF, 3NF and BCNF with a running example, showing how each anomaly is removed at each stage.
5. Explain all types of attributes in the E-R model (simple, composite, single-valued, multi-valued, derived, key) with a suitable entity example and diagram description.
6. Explain all types of Integrity Constraints in DBMS (Domain, Entity, Referential) and all SQL-level constraints (NOT NULL, UNIQUE, CHECK, DEFAULT, PRIMARY KEY, FOREIGN KEY) with examples.

Multiple Choice Questions (MCQs)

1. Which of the following is NOT a level in the ANSI-SPARC three-level architecture?

a. External Level
b. Conceptual Level
c. Network Level
d. Internal Level

Answer: (c) Network Level

2. Who proposed the Relational Data Model?

a. Peter Chen
b. E. F. Codd
c. Charles Bachman
d. Edgar Dijkstra

Answer: (b) E. F. Codd

3. A weak entity is identified using:

a. Its own primary key only
b. A foreign key alone
c. Its partial key + the owner entity's primary key
d. None of the above

Answer: (c) Its partial key + the owner entity's primary key

4. Which normal form removes transitive dependency?

a. 1NF
b. 2NF
c. 3NF
d. None

Answer: (c) 3NF

5. The number of tuples in a relation is called its:

a. Degree
b. Cardinality
c. Domain
d. Schema

Answer: (b) Cardinality

6. In an E-R diagram, a diamond shape represents:

a. Entity
b. Attribute
c. Relationship
d. Key

Answer: (c) Relationship

7. Which constraint ensures a foreign key value matches an existing primary key value or is NULL?

a. Domain constraint
b. Entity Integrity constraint
c. Referential Integrity constraint
d. Check constraint

Answer: (c) Referential Integrity constraint

8. Which data model allows a record to have more than one parent?

a. Hierarchical Model
b. Network Model
c. Relational Model
d. None

Answer: (b) Network Model

9. Physical data independence means:

a. Changing external view without affecting conceptual view
b. Changing conceptual view without affecting external view
c. Changing internal/storage structure without affecting the conceptual schema
d. None of the above

Answer: (c) Changing internal/storage structure without affecting the conceptual schema

10. A composite primary key made of two attributes means:

a. Two separate primary keys exist
b. Both attributes together uniquely identify a row
c. Neither attribute can repeat individually
d. It is invalid in relational databases

Answer: (b) Both attributes together uniquely identify a row

MODULE 2

Relational Query Languages

Syllabus: Relational algebra, Tuple and domain relational calculus, SQL3, DDL and DML constructs, open source and commercial DBMS (MySQL, ORACLE, DB2, SQL Server).

1. Introduction — Query Languages

A **query language** lets users retrieve and manipulate data stored in a relational database. Relational query languages are broadly classified as:

Type	Meaning	Examples
Procedural	User specifies *what* data is needed AND *how* (the step-by-step sequence of operations) to get it	Relational Algebra
Non-Procedural (Declarative)	User specifies only *what* data is needed, not how to retrieve it — the DBMS decides the execution plan	Relational Calculus, SQL

2. Relational Algebra

Relational Algebra is a **procedural query language** that takes one or two relations as input and produces a new relation as output, using a set of well-defined operations. It is the formal, mathematical foundation on which SQL is built.

2.1 Sample Tables used throughout this section

STUDENT

Roll_No	Name	Dept	Age
1	Aman	CSE	20
2	Priya	ECE	21
3	Rohan	CSE	19
4	Sara	IT	22

DEPARTMENT

Dept	HOD
CSE	Dr. Rao
ECE	Dr. Sen
ME	Dr. Iyer

2.2 Fundamental (Basic) Operations

(a) SELECT (σ) — Selects Rows

Picks tuples (rows) that satisfy a given condition. Notation: `σ<condition>(Relation)`

Query: `σ Dept='CSE' (STUDENT)` — get all CSE students.
Result:

Roll_No	Name	Dept	Age
1	Aman	CSE	20
3	Rohan	CSE	19

(b) PROJECT (π) — Selects Columns

Picks specified columns (attributes), removing duplicate rows. Notation: `π<attribute-list>(Relation)`

Query: `π Name, Dept (STUDENT)` — list only names and departments.
Result:

Name	Dept
Aman	CSE
Priya	ECE
Rohan	CSE
Sara	IT

(c) UNION (∪)

Combines tuples from two **union-compatible** relations (same number/type of attributes), removing duplicates.

Query: `π Dept(STUDENT) ∪ π Dept(DEPARTMENT)` — all departments that either have a student or exist in the Department table: {CSE, ECE, IT, ME}.

(d) SET DIFFERENCE (−)

Returns tuples present in the first relation but NOT in the second (relations must be union-compatible).

Query: `π Dept(DEPARTMENT) − π Dept(STUDENT)` — departments with no students at all: {ME}.

(e) CARTESIAN PRODUCT (×)

Combines every tuple of relation A with every tuple of relation B (all possible pairings) — result has (rows A × rows B) tuples.

Query: `STUDENT × DEPARTMENT` — produces 4 × 3 = 12 rows pairing every student with every department (most pairings are meaningless until filtered by a condition — this is exactly why JOIN exists).

(f) RENAME (ρ)

Renames a relation and/or its attributes, without changing the data — useful for self-joins or clarity. Notation: `ρ NewName(OldRelation)`

Query: `ρ S(STUDENT)` — refers to STUDENT as "S" in the rest of the query.

2.3 Derived (Additional) Operations

These can all be expressed using the fundamental operations above, but are given their own notation for convenience.

(a) INTERSECTION (∩)

Returns tuples that appear in BOTH relations. (R ∩ S = R − (R − S))

Query: If relation A = {CSE persons} and relation B = {Scholarship holders}, then `A ∩ B` gives CSE students who also hold scholarships.

(b) JOIN (⋈)

Join Type	Meaning
Theta Join (⋈θ)	Combines tuples of two relations that satisfy a general condition θ (e.g. >, <, =)
Equi-Join	A theta join where the condition uses only equality (=)
Natural Join (⋈)	An equi-join on all common attribute names, with the duplicate column automatically removed

Query: `STUDENT ⋈ DEPARTMENT` (Natural Join on Dept) —

Roll_No	Name	Dept	Age	HOD
1	Aman	CSE	20	Dr. Rao
2	Priya	ECE	21	Dr. Sen
3	Rohan	CSE	19	Dr. Rao

(Sara from IT is dropped because IT has no matching row in DEPARTMENT — this is exactly equivalent to `σ STUDENT.Dept=DEPARTMENT.Dept (STUDENT × DEPARTMENT)` followed by removing the duplicate Dept column.)

(c) DIVISION (÷)

Used when a query needs "all" — e.g. finds tuples in R that are associated with **every** tuple in S.

Real-life example: Given ENROLLED(Student, Course) and a relation ALL_CORE_COURSES(Course), the query `ENROLLED ÷ ALL_CORE_COURSES` returns students who have enrolled in **every** core course — e.g. "find students who have taken all 4 compulsory subjects."

3. Relational Calculus

Relational Calculus is a **non-procedural (declarative)** query language — the user describes **what** result is wanted (using logical predicates/conditions), not the step-by-step method to compute it. It has two forms:

3.1 Tuple Relational Calculus (TRC)

Variables range over **tuples** of a relation. General form: `{ t | P(t) }` — meaning "the set of all tuples t such that predicate P(t) is true."

Query: Get names and departments of all students in CSE:
`{ t.Name, t.Dept | t ∈ STUDENT ∧ t.Dept = 'CSE' }`

3.2 Domain Relational Calculus (DRC)

Variables range over **individual domain values** (single attribute values) instead of whole tuples. General form: `{ <x1, x2, …, xn> | P(x1, x2, …, xn) }`

Query: Same query in DRC:
`{ <name, dept> | ∃ roll,age ( <roll, name, dept, age> ∈ STUDENT ∧ dept = 'CSE' ) }`

4. Relational Algebra vs Relational Calculus

Basis	Relational Algebra	Relational Calculus
Nature	Procedural language — specifies the sequence of operations	Non-procedural (declarative) language — specifies only the desired result
Approach	How to obtain the result (step by step)	What result is wanted (logic/predicate based)
Basis	Based on set theory and algebraic operations (σ, π, ∪, ×, ⋈, …)	Based on predicate/first-order logic
Variables	Operates directly on relations	Uses tuple variables (TRC) or domain variables (DRC)
Ease of use	Closer to programming — step-by-step, more explicit	Closer to natural/mathematical language — more abstract
Expressive Power	Equivalent in power to *safe* relational calculus (Codd's theorem)	Equivalent in power to relational algebra
Example	`π Name(σ Dept='CSE'(STUDENT))`	`{ t.Name | t ∈ STUDENT ∧ t.Dept='CSE' }`

5. SQL (Structured Query Language)

SQL is the standard **non-procedural** language used to create, manipulate and control relational databases (MySQL, Oracle, DB2, SQL Server all understand SQL, with minor vendor differences). SQL commands are grouped into four categories:

Category	Full Form	Purpose	Commands
DDL	Data Definition Language	Defines/modifies the structure (schema) of database objects	CREATE, ALTER, DROP, TRUNCATE
DML	Data Manipulation Language	Inserts, updates, deletes and retrieves data	INSERT, UPDATE, DELETE, SELECT
DCL	Data Control Language	Controls access/permissions on data	GRANT, REVOKE
TCL	Transaction Control Language	Manages transactions (groups of operations)	COMMIT, ROLLBACK, SAVEPOINT

5.1 DDL — Data Definition Language

(a) CREATE TABLE — builds a new table

```
CREATE TABLE Student (
    Roll_No INT PRIMARY KEY,
    Name    VARCHAR(30) NOT NULL,
    Dept    VARCHAR(10),
    Age     INT CHECK (Age >= 18)
);
```

Effect — a brand-new, empty table structure is created:

Roll_No	Name	Dept	Age
(no rows yet — only column structure exists)			

(b) ALTER TABLE — modifies an existing table's structure

```
ALTER TABLE Student ADD Email VARCHAR(40);
ALTER TABLE Student DROP COLUMN Age;
ALTER TABLE Student MODIFY Name VARCHAR(50);
```

Before:

Roll_No	Name	Dept
1	Aman	CSE

After ADD Email:

Roll_No	Name	Dept	Email
1	Aman	CSE	NULL

(c) DROP TABLE — permanently deletes a table (structure + all data)

```
DROP TABLE Student;
```

The entire table disappears from the database — irreversible.

(d) TRUNCATE TABLE — deletes all rows but keeps the table structure

```
TRUNCATE TABLE Student;
```

Before: 4 rows present. **After TRUNCATE:** table Student still exists with the same columns, but 0 rows — much faster than DELETE for wiping an entire table since it does not log individual row deletions.

5.2 DML — Data Manipulation Language

(a) INSERT — adds a new row

```
INSERT INTO Student (Roll_No, Name, Dept, Age)
VALUES (1, 'Aman', 'CSE', 20);
```

Before: (empty) **After:**

Roll_No	Name	Dept	Age
1	Aman	CSE	20

(b) SELECT — retrieves data

```
SELECT Name, Dept FROM Student WHERE Age > 19;
```

Table:

Roll_No	Name	Dept	Age
1	Aman	CSE	20
3	Rohan	CSE	19

Result (Age > 19 filters out Rohan):

Name	Dept
Aman	CSE

(c) UPDATE — modifies existing rows

```
UPDATE Student SET Dept = 'IT' WHERE Roll_No = 1;
```

Before:

Roll_No	Name	Dept
1	Aman	CSE

After:

Roll_No	Name	Dept
1	Aman	IT

(d) DELETE — removes existing rows

```
DELETE FROM Student WHERE Roll_No = 1;
```

Before: 1 row present (Roll_No=1). **After:** that row is gone (table structure remains, unlike DROP).

5.3 DCL — Data Control Language

```
GRANT SELECT, INSERT ON Student TO 'clerk_user';
```

```
REVOKE INSERT ON Student FROM 'clerk_user';
```

Real-life example: The Library clerk's login is `GRANT` ed SELECT and UPDATE rights on the "Books" table but NOT on the "Fee" table; later, if the clerk changes role, the DBA can `REVOKE` those rights.

5.4 TCL — Transaction Control Language

```
BEGIN TRANSACTION;
UPDATE Account SET Balance = Balance - 5000 WHERE Acc_No = 'A1';
UPDATE Account SET Balance = Balance + 5000 WHERE Acc_No = 'A2';
COMMIT;       -- makes changes permanent
-- ROLLBACK;  -- would undo both updates if something failed
SAVEPOINT sp1;  -- a checkpoint inside a long transaction to roll back to partially
```

Real-life example: A bank fund transfer debits ₹5000 from Account A1 and credits it to A2. `COMMIT` saves both changes permanently only if both succeed; if the system crashes after the debit but before the credit, `ROLLBACK` undoes the debit too — preventing money from vanishing (this is the **Atomicity** property of transactions).

6. SQL Clauses, Joins & Aggregate Functions

6.1 Common Clauses

Clause	Purpose	Example
WHERE	Filters rows by a condition	`SELECT * FROM Student WHERE Dept='CSE';`
DISTINCT	Removes duplicate rows from the result	`SELECT DISTINCT Dept FROM Student;`
ORDER BY	Sorts the result (ASC/DESC)	`SELECT * FROM Student ORDER BY Age DESC;`
GROUP BY	Groups rows sharing a value, for use with aggregate functions	`SELECT Dept, COUNT(*) FROM Student GROUP BY Dept;`
HAVING	Filters groups (used only after GROUP BY — WHERE cannot filter aggregates)	`... GROUP BY Dept HAVING COUNT(*) > 1;`

GROUP BY example — count students per department:

Roll_No	Name	Dept
1	Aman	CSE
2	Priya	ECE
3	Rohan	CSE

Query: `SELECT Dept, COUNT(*) AS Total FROM Student GROUP BY Dept;`

Dept	Total
CSE	2
ECE	1

6.2 Aggregate Functions

Function	Purpose
COUNT()	Number of rows
SUM()	Total of a numeric column

AVG()	Average value
MAX() / MIN()	Highest / lowest value

6.3 SQL Joins

STUDENT

Roll_No	Name	Dept
1	Aman	CSE
2	Priya	ECE
4	Sara	IT

DEPARTMENT

Dept	HOD
CSE	Dr. Rao
ECE	Dr. Sen
ME	Dr. Iyer

Join Type	SQL	Result (on Dept)
INNER JOIN only matching rows on both sides	`SELECT * FROM Student INNER JOIN Department ON Student.Dept=Department.Dept;`	Aman-CSE-Dr.Rao, Priya-ECE-Dr.Sen (Sara/IT and ME dropped — no match)
LEFT JOIN all rows from left (Student), matched or NULL	`SELECT * FROM Student LEFT JOIN Department ON Student.Dept=Department.Dept;`	Aman-CSE-Dr.Rao, Priya-ECE-Dr.Sen, Sara-IT-**NULL**
RIGHT JOIN all rows from right (Department), matched or NULL	`SELECT * FROM Student RIGHT JOIN Department ON Student.Dept=Department.Dept;`	Aman-CSE-Dr.Rao, Priya-ECE-Dr.Sen, **NULL**-ME-Dr.Iyer
FULL OUTER JOIN all rows from both sides, matched or NULL	`SELECT * FROM Student FULL OUTER JOIN Department ON Student.Dept=Department.Dept;`	All 4 rows: CSE match, ECE match, Sara-IT-NULL, NULL-ME-Dr.Iyer

6.4 Query Processing & Optimization (Overview)

When SQL (non-procedural) is submitted, the DBMS's **query optimizer** translates it internally into an efficient sequence of relational algebra operations, using rules such as: perform SELECT (σ) and PROJECT (π) as early as possible (to shrink intermediate relation sizes before an expensive JOIN), choose the cheapest available join strategy, and use indexes where they exist — guided by Armstrong's Axioms/FD analysis and cost estimation, so that logically-equivalent expressions are compared and the lowest-cost execution plan is chosen.

7. Module 2 — Practice Questions

2-Mark Questions

1. Define Relational Algebra. Is it procedural or non-procedural?
2. What is the difference between DDL and DML? Give one command of each.
3. What is the purpose of the RENAME (ρ) operation?
4. Differentiate between DELETE and TRUNCATE.
5. What is the purpose of the HAVING clause?
6. Define Tuple Relational Calculus.
7. What is the difference between WHERE and HAVING clauses?
8. Name the four categories of SQL commands.

4-Mark Questions

1. Explain the SELECT and PROJECT operations of relational algebra with an example.
2. Explain Natural Join and Theta Join with a suitable example.
3. Differentiate between Relational Algebra and Relational Calculus.
4. Explain INNER JOIN, LEFT JOIN and RIGHT JOIN with an example.
5. Explain the DCL and TCL commands with examples.
6. Explain Tuple Relational Calculus and Domain Relational Calculus with an example each.
7. Write SQL queries to CREATE a table, INSERT two rows, and SELECT rows with a condition.
8. Explain the CARTESIAN PRODUCT operation and how JOIN improves upon it.

8-Mark Questions

1. Explain all fundamental operations of Relational Algebra (Select, Project, Union, Set Difference, Cartesian Product, Rename) with suitable table examples.
2. Explain all DDL, DML, DCL and TCL commands of SQL with syntax and example tables showing before/after effect.
3. Differentiate between Relational Algebra and Relational Calculus in detail, covering nature, approach, expressive power and examples.
4. Explain all types of SQL Joins (INNER, LEFT, RIGHT, FULL OUTER) with sample tables and result sets.
5. Write short notes on: (a) GROUP BY and HAVING clause (b) Aggregate functions (c) Division operation in relational algebra, each with an example.

Multiple Choice Questions (MCQs)

1. Which relational algebra operation selects rows based on a condition?

a. π (Project)
b. σ (Select)
c. ρ (Rename)
d. × (Cartesian Product)

Answer: (b) σ (Select)

2. Which SQL command is used to permanently save transaction changes?

a. ROLLBACK
b. SAVEPOINT
c. COMMIT
d. GRANT

Answer: (c) COMMIT

3. TRUNCATE differs from DELETE because:

a. TRUNCATE removes the table structure too
b. TRUNCATE removes all rows quickly and cannot use a WHERE clause

c. DELETE cannot remove rows
d. There is no difference

Answer: (b) TRUNCATE removes all rows quickly and cannot use a WHERE clause

4. Which of the following is a non-procedural query language?

a. Relational Algebra
b. Relational Calculus
c. C Programming
d. None

Answer: (b) Relational Calculus

5. Which JOIN returns all rows from both tables, matched or not?

a. INNER JOIN
b. LEFT JOIN
c. RIGHT JOIN
d. FULL OUTER JOIN

Answer: (d) FULL OUTER JOIN

6. Which clause is used to filter grouped results?

a. WHERE
b. GROUP BY
c. HAVING
d. ORDER BY

Answer: (c) HAVING

7. GRANT and REVOKE belong to which category of SQL commands?

a. DDL
b. DML
c. DCL
d. TCL

Answer: (c) DCL

8. The Natural Join operation automatically:

a. Removes duplicate rows only
b. Joins on all common attribute names and removes the duplicate column
c. Performs a Cartesian Product only
d. Sorts the result

Answer: (b) Joins on all common attribute names and removes the duplicate column

9. Which relational algebra operator is used to find students enrolled in ALL core courses?

a. Union
b. Intersection
c. Division
d. Join

Answer: (c) Division

10. In Tuple Relational Calculus, the variable ranges over:

a. Individual domain values
b. Whole tuples of a relation
c. Table names
d. SQL keywords

Answer: (b) Whole tuples of a relation

— End of Notes (Module 1 & Module 2) —